

Introducción a la Programación I

Clase 3: Estructuras Condicionales (if)

¿Para Qué Sirven las Condicionales?

Los programas necesitan **tomar decisiones**: ejecutar diferente código según las circunstancias.

Ejemplos del mundo real:

- Si el usuario es mayor de edad → mostrar contenido
- Si la contraseña es correcta → iniciar sesión
- Si el saldo es suficiente → aprobar el pago

En Python usamos sentencias `if` para esto.

Ejemplo Simple

```
edad = 20  
  
if edad >= 18:  
    print("Sos mayor de edad.")
```

Sos mayor de edad.

Si `edad` fuera `15`, no se imprimiría nada.

Tests Condicionales

Un **test condicional** es una expresión que se evalúa como `True` o `False`.

```
auto = "toyota"

print(auto == "toyota")    # True
print(auto == "honda")    # False
```

`=` es asignación, `==` es comparación.

Igualdad y Desigualdad

```
# Igualdad
print(5 == 5)          # True
print("gato" == "gato") # True

# Desigualdad
print(5 != 3)          # True
print("gato" != "perro") # True
```

Con strings, la comparación distingue mayúsculas:

```
print("Python" == "python") # False
print("Python".lower() == "python") # True
```

Comparaciones Numéricas

Operador	Significado	Ejemplo
<	menor que	3 < 5 → True
<=	menor o igual	5 <= 5 → True
>	mayor que	7 > 2 → True
>=	mayor o igual	4 >= 6 → False

```
temperatura = 37.5
```

```
print(temperatura > 37)    # True  
print(temperatura >= 38)   # False
```

Chequear Múltiples Condiciones: **and**

and es **True** solo cuando **ambas** condiciones son **True** :

```
edad = 25
tiene_licencia = True

if edad >= 18 and tiene_licencia:
    print("Podés manejar.")
```

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

Chequear Múltiples Condiciones: **or**

or es **True** cuando **al menos una** condición es **True** :

```
dia = "sábado"

if dia == "sábado" or dia == "domingo":
    print("Es fin de semana.")
```

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

Paréntesis para Mayor Legibilidad

Al combinar `and` y `or`, los paréntesis ayudan a que el código sea más claro:

```
edad = 22
tiene_dni = True
tiene_pasaporte = False

# Sin paréntesis – ¿qué se evalúa primero?
if edad >= 18 and tiene_dni or tiene_pasaporte:
    pass

# Con paréntesis – intención clara
if edad >= 18 and (tiene_dni or tiene_pasaporte):
    print("Puede ingresar.")
```

Expresiones y Variables Booleanas

El tipo `bool` tiene solo dos valores: `True` y `False`.

```
juego_activo = True  
partida_ganada = False
```

Se usan directamente en condicionales:

```
if juego_activo:  
    print("El juego está en curso.")  
  
if not partida_ganada:  
    print("Seguí intentando.")
```

`not` invierte el valor booleano.

Sentencia `if` Simple

Sintaxis:

```
if condicion:  
    # código que se ejecuta si la condición es True
```

Ejemplo:

```
puntaje = 95  
  
if puntaje >= 90:  
    print("¡Excelente!")
```

La indentación (4 espacios) define el bloque de código del `if`.

Sentencia **if-else**

Cuando querés ejecutar código tanto si la condición es **True** como si es **False** :

```
edad = 16

if edad >= 18:
    print("Podés votar.")
else:
    print("Todavía no podés votar.")
```

Todavía no podés votar.

Cadena `if-elif-else`

Para elegir entre más de dos opciones:

```
edad = 12

if edad < 4:
    precio = 0
elif edad < 18:
    precio = 500
elif edad < 65:
    precio = 1200
else:
    precio = 800

print(f"El precio de la entrada es: ${precio}")
```

El precio de la entrada es: \$500

Cómo Funciona `if-elif-else`

Python evalúa las condiciones **de arriba hacia abajo**.

En cuanto una condición es `True`, ejecuta ese bloque y **sale de la cadena**.

```
x = 5

if x > 0:
    print("positivo")      # ← se ejecuta esto
elif x > 3:
    print("mayor que 3")  # ← esto no se evalúa
else:
    print("cero o negativo")
```

positivo

Múltiples Bloques `elif`

Podés tener tantos `elif` como necesites:

```
nota = 75

if nota >= 90:
    print("Sobresaliente")
elif nota >= 80:
    print("Notable")
elif nota >= 70:
    print("Bueno")
elif nota >= 60:
    print("Aprobado")
else:
    print("Desaprobado")
```

Bueno

Omitir el Bloque `else`

El bloque `else` es **opcional**. A veces no hace falta un caso por defecto:

```
dia = "lunes"

if dia == "sábado":
    print("Fin de semana")
elif dia == "domingo":
    print("Fin de semana")
```

Si `dia` es `"lunes"`, no se ejecuta nada — y eso está bien.

Múltiples `if` Independientes

A veces querés chequear **todas las condiciones** por separado, no elegir una sola:

```
ingredientes = ["queso", "tomate", "aceitunas"]

if "queso" in ingredientes:
    print("Llevar extra queso.")

if "tomate" in ingredientes:
    print("Pedir menos tomate.")

if "piña" in ingredientes:
    print("Repensar las decisiones de vida.")
```

Con `if-elif` solo se ejecutaría el primero que sea `True`. Con `if` separados, se evalúan **todos**.

Resumen de la Clase

Concepto	Sintaxis
<code>if</code>	<code>if condicion:</code>
<code>if-else</code>	<code>if ...: ... else: ...</code>
<code>if-elif-else</code>	Múltiples condiciones en cadena
<code>and</code>	Ambas condiciones deben ser True
<code>or</code>	Al menos una debe ser True
<code>not</code>	Invierte el valor booleano
Variables bool	<code>activo = True</code>
Comparaciones	<code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>